

Разработка клиент-серверных приложений

Лекция 2

Стили и форматирование.

Каскадные таблицы стилей. Cascading Style Sheets (CSS).

Виды верстки.

История. HTML: атрибуты

Я **очень** люблю:

- компьютеры
- кошек (*иногда*)

Я
очень
люблю:
...

Тег

Атрибут

Недостаток: смешивание содержимого и стиля отображения.

HTML5: Элемент **font** считается устаревшим.

HTML: стили

Я **очень** люблю:

- компьютеры
- кошек (*иногда*)

Я `<b style="font-size:20pt; color:red">`

очень`</b>`

люблю:

...

Стиль

HTML: стилевые файлы

Page.html

```
Я <b class="megared">очень</b>  
люблю:  
...
```

Styles.css

```
.megared {  
    font-size: 20pt;  
    color: red;  
}
```

Каскадные таблицы стилей

Такие стили называются «каскадными», поскольку для любого стиля можно переопределять отдельные атрибуты.

Стили «наследуются» в порядке появления элементов на странице и могут объявляться:

- во внешних файлах CSS: `<link rel="stylesheet" type="text/css" href="..." />`
- внутри страницы (тег style): `<style>...</style>`
- внутри тега (атрибут style): `<span style="..."></span>`

Возможно переопределять форматирование:

- С явным указанием класса: `.class { ... }`
- Для всех тегов: `h1 { color: red; }`
- Для заданного id тега: `#id { ... }`

Каскадные таблицы стилей: основная идея

Стили – это гибкий способ переопределять форматирование элементов на странице.

Самое главное:

- HTML-разметка определяет содержимое (контент);
- CSS-разметка определяет внешний вид (дизайн).

Заменой стилей можно легко менять отображение одной и той же информации для различных устройств!

Каскадные таблицы стилей: селекторы и объявления

- ✓ **CSS (Cascading Style Sheets)**, или каскадные таблицы стилей, описывают правила форматирования отдельного элемента веб-страницы.
- ✓ Создав стиль один раз, его можно применять к любым элементам страницы сколько угодно раз.
- ✓ Определение стиля состоит из двух основных частей: самого элемента веб-страницы – **селектора**, и команды форматирования – **блока объявления**.
- ✓ Селектор сообщает браузеру, какой именно элемент форматировать, в блоке объявления перечисляются формирующие команды.



Каскадные таблицы стилей: способы добавления стилей

```
<div style="width:200px; color: red;">
```

Информация

```
</div>
```

✓ **Внутритекстовые стили**

```
<head>
```

```
<style>
```

```
.box{
```

```
width:200px;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="box"> Информация </div>
```

```
</body>
```

✓ **Встраиваемые стили**

```
<head>
```

```
<link rel="stylesheet" type="text/css" href="style/style.css">
```

```
</head>
```

✓ **Внешняя таблица стилей**

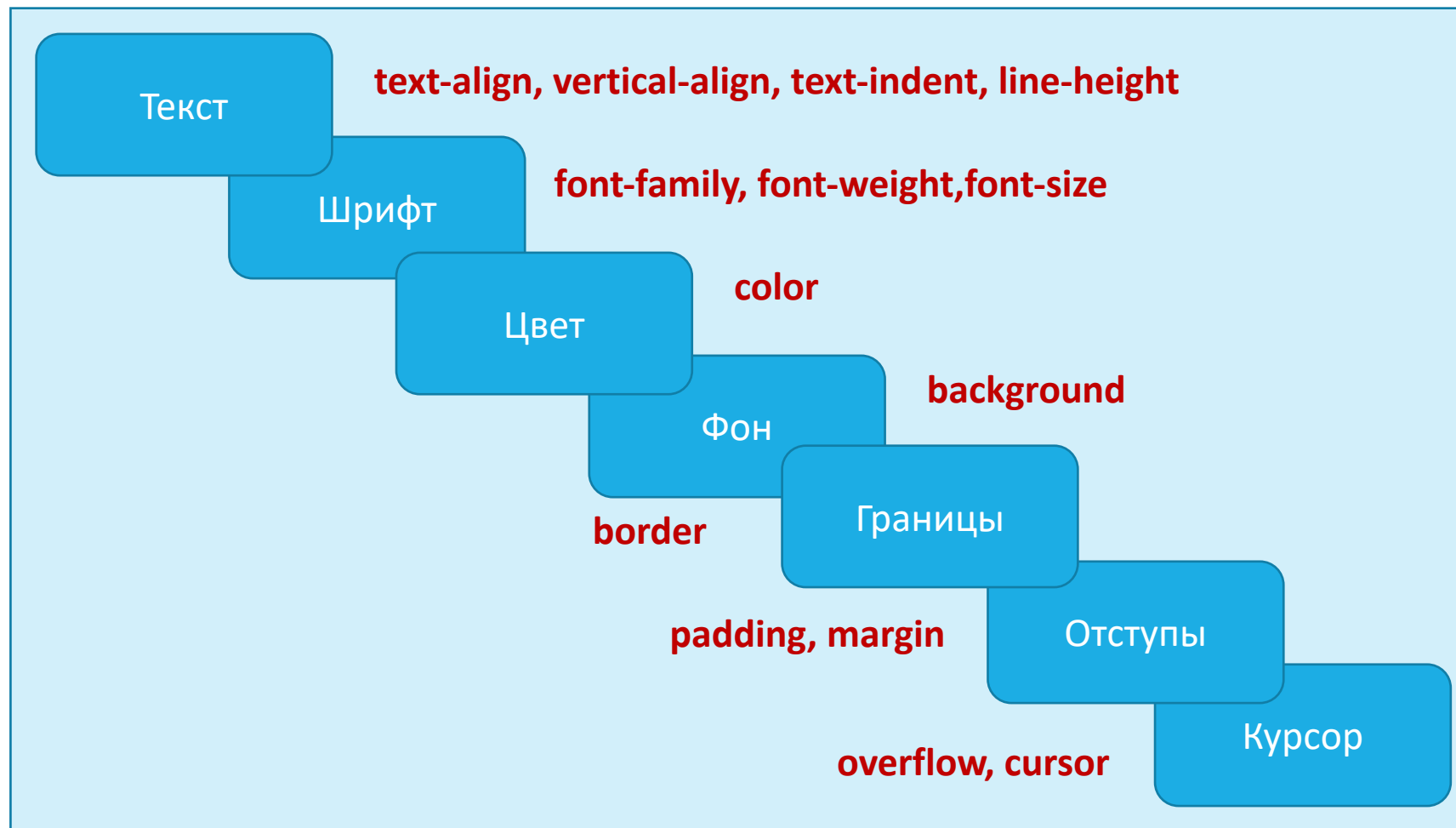
Принципы наследования и каскадирования

- ✓ **Принцип наследования** заключается в том, что свойства CSS, объявленные для элементов-предков, наследуются элементами потомками.
- ✓ **Принцип каскадирования** представляет собой процесс применения различных правил к одному и тому же элементу. Более конкретные правила имеют приоритет над более общими. Если в отношении одного и того же элемента определено несколько стилей, то в результате к нему будет применен последний из них.

Приоритеты

- ✓ Самым низким приоритетом обладают стили, заданные в браузере по умолчанию и наследуемые в документе элементом от своих предков.
- ✓ Более высоким приоритетом обладают стили, заданные во внешних таблицах стилей, подключённых к документу.
- ✓ Ещё более высоким приоритетом обладают стили, заданные непосредственно селекторами, содержащимися в контейнерах (узлах) style данного документа.
- ✓ Затем приоритетом обладают стили, объявленные непосредственно в теге данного элемента посредством атрибута style этого тега.
- ✓ И наконец самым высоким приоритетом обладают стили, объявленные автором страницы *или пользователем*, с помощью директивы !important.

Базовые стили



Стили для текста

Горизонтальное выравнивание: `text-align`

Отступ: `text-indent`

Высота строк: `line-height`

Вертикальное выравнивание: `vertical-align`

Расстояние между словами: `word-spacing`

Расстояние между буквами: `letter-spacing`

Обработка пробелов: `white-space`

Настройка табуляции: `tab-size`

. Преобразование текста `text-transform`

. Направление написания текста `direction`

. Направление написания слов в тексте `unicode-bidi`

. Оформление текста `text-decoration`

. Форматирование первой буквы и первой строки абзаца `:first-letter` и `:first-line`

. Кавычки `quotes`

CSS текст представляет набор css-стилей для форматирования текстового содержимого веб-страниц.

Стили для шрифтов

CSS шрифты управляют внешним видом шрифта текста веб-страниц.

Семейство шрифтов `font-family`

Стиль шрифта `font-style`

Варианты шрифтов `font-variant`

Насыщенность шрифта `font-weight`

Размер шрифта `font-size`

Цвет шрифта `color`

- ✓ Текст основного содержимого веб-страницы должен быть в первую очередь читабельным.
- ✓ Не рекомендуется использовать более двух шрифтов на странице.

Стили фона

1. Цвет изображения `background-color`
2. Фоновое изображение `background-image`
3. Повтор фоновых изображений `background-repeat`
4. Позиционирование фоновых изображений `background-position`
5. Фиксация изображения на месте `background-attachment`
6. Заполнение фоном отступов и границ элемента `background-clip`
7. Положение фонового изображения относительно его родительского блока `background-origin`
8. Размер изображения `background-size`
9. Задание фона элемента одним свойством `background`



Стили для рамки (границ элемента)

1. Стил ь рамки `border-style`

2. Цвет рамки `border-color`

3. Ширина рамки `border-width`

4. Задание рамки одним свойством `border`

5. Задание рамки для одной границы элемента `border-top`, `border-bottom`, `border-left`, `border-right`

- ✓ **CSS рамка** задается с помощью краткого свойства `border`
- ✓ Стил ь рамки задается с помощью трех свойств:
стил ь, цвет и ширина.

Тень текста

1. #1 { text-shadow:rgba(0,0,0,0.5) 1px 1px 0; }
2. #2 { text-shadow:rgba(0,0,0,0.7) 5px 5px 3px; }
3. #3 { text-shadow:rgba(45,35,200,0.7) -10px -10px 3px; }

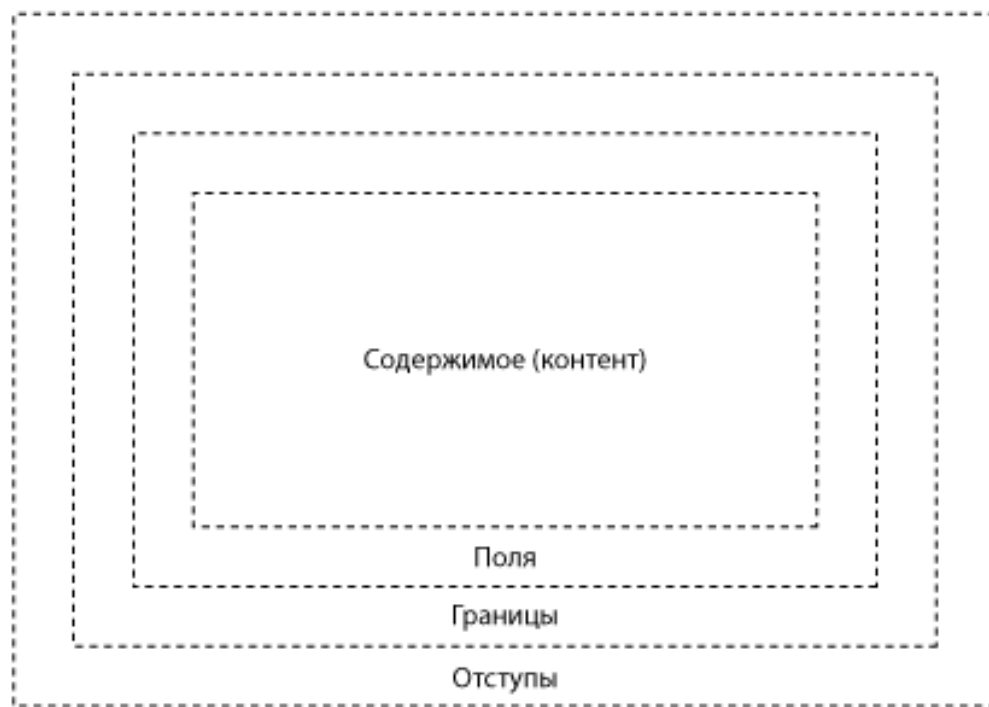
1 Lorem ipsum dolor sit amet, consectetur adipiscing elit

2 Lorem ipsum dolor sit amet, consectetur adipiscing elit

3 Lorem ipsum dolor sit amet, consectetur adipiscing elit

Блочная модель

- Позиционирование блоков: `position`; `left`, `top`, `bottom`, `right`; `z-index` и т.д.
- Размеры, поля, границы, отступы блочных (и некоторых строчных) элементов: `height`, `width`; `margin`; `border`; `padding`;
- Обтекание и обрезка контента: `float`; `clear`; `overflow`;



Цвета и прозрачность

С помощью шести значений — по два на красный, зеленый и синий — все цвета могут быть заданы посредством шести шестнадцатеричных целых чисел (#6F001C == Красный: 6F, Зеленый: 00, Синий: 1C.).

Для описания **уровня непрозрачности** при меняется свойство **альфа**, значения которого находятся в промежутке между **0 и 1**.

`rgba (255, 0, 0, 0,5)` ← Добавление прозрачности

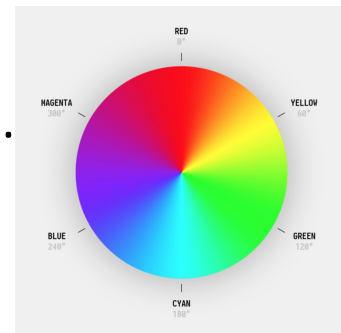
генерирует красный цвет с 50-процентной непрозрачностью.

Тон-насыщенность-яркость (HSL, Hue-Saturation-Light).

`hsla (120, 100%, 50%, 0,3)`

дает зеленый цвет с 30-процентной непрозрачностью (или 70-процентной прозрачностью).

Оттенок (360%) насыщенность (%) светлота



Стили для отступов



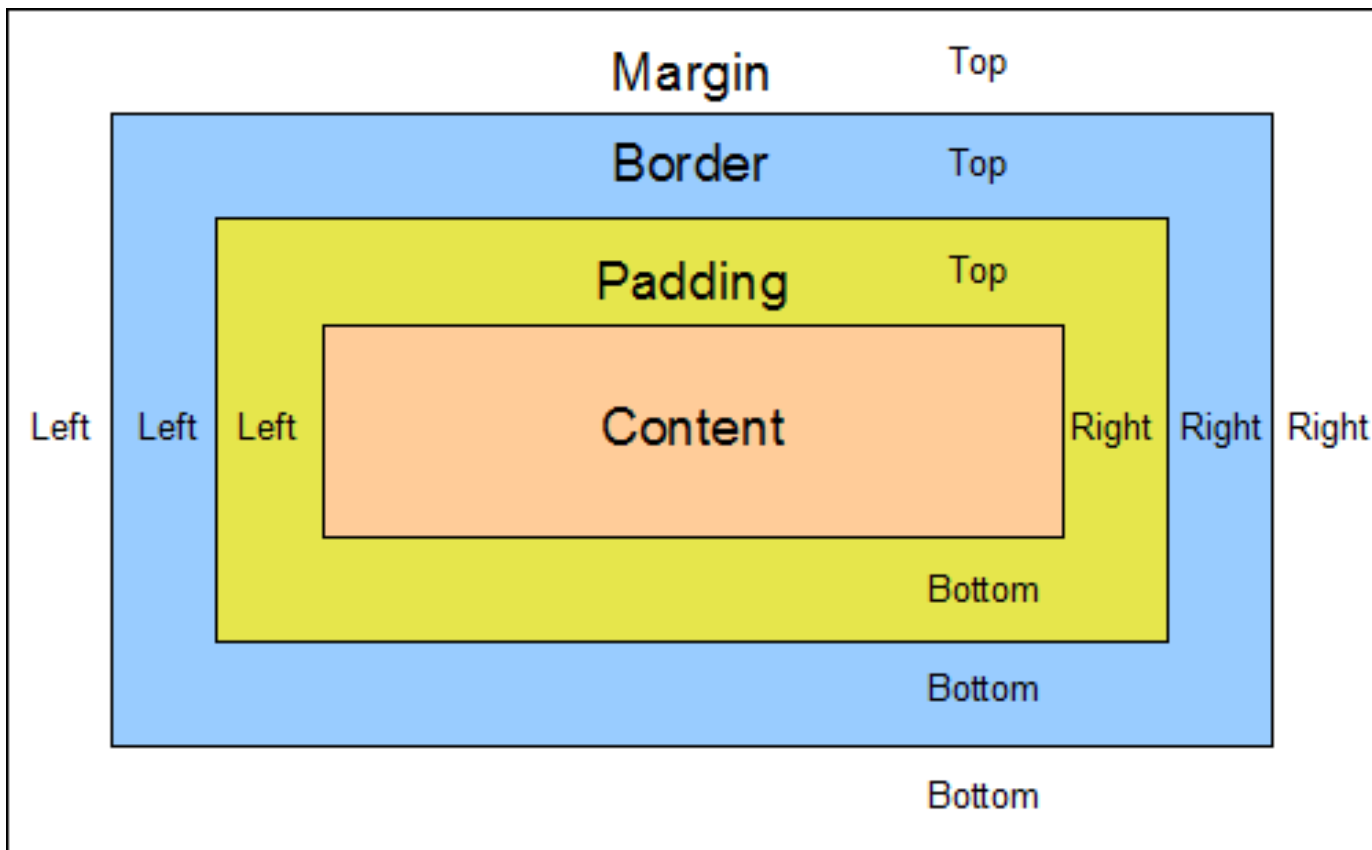
- ✓ **Область содержимого** – это содержимое элемента.
- ✓ **Внешний отступ (margin)** добавляет отступы за границами элемента, создавая тем самым промежутки между элементами.
- ✓ Они всегда остаются прозрачными и через них виден фон родительского элемента.
- ✓ Значения padding и margin задаются в следующем порядке: верхнее, правое, нижнее и левое.

- ✓ **Внутренний отступ**, или поле элемента, (**padding**) добавляет отступы внутри элемента, между его основным содержимым и его границей. Если для элемента задать фон, то он распространится также и на поля элемента. Внутренний отступ не может принимать отрицательных значений, в отличие от внешнего отступа.

Стили: спецификация отступов со всех сторон



padding: **Top Right Bottom Left**;



padding: 5px 5px 5px 5px; > padding: 5px;

padding: 5px 9px 5px 9px; > padding: 5px 9px;

Стили: спецификация отступов с одной из сторон

padding-top: 5px;
padding-right: 6px;
padding-bottom: 7px;
padding-left: 8px;

padding: 5px 6px 7px 8px;

Примеры правильного/неправильного указания значений:

padding: 5px; +

padding: 0px; +

padding: 5; -

padding: 0; +

Псевдоэлементы и псевдоклассы

- ✓ **Псевдоклассы** – это селекторы, которые определяют состояние уже существующих элементов, которое может меняться при определенных условиях (например, `:first-child`, `:last-child`, `:hover`, `:focus`, `:link`, `:active`, `:visited`, `:lang`).
- ✓ **Псевдоэлементы** – это селекторы, которые определяют область элементов, которая изначально отсутствует в дереве документа. Эта область создается искусственно с помощью CSS (например, `:before`, `:after`, `:first-line`, `:first-letter`).
- ✓ Ключевое отличие: **псевдоклассы** определяют состояние элементов, которые уже существуют на странице, а **псевдоэлементы** создают области (искусственные элементы), которых изначально на веб-странице не было. Но и те и другие отсутствуют в исходном коде документа.
- ✓ Другими словами: псевдокласс задает стиль для **элемента страницы** в определенном состоянии, а псевдоэлемент задаёт стиль для **части элемента страницы** и даже может создавать дополнительную часть.

Примеры псевдоэлементов

after - добавление контента ПОСЛЕ указанного элемента

before - добавление контента ДО указанного элемента

firstletter - стили для первой буквы в контенте элемента

firstline - стилевое оформление первой строки текста в элементе

selection - применение стилей при выделении текста в элементе

- ✓ Одной из самых распространённых задач является добавление фразы до или после элемента.
- ✓ Псевдоэлементы **after** и **before** предназначены для "врезки" в страницу сайта контента который изначально не указан в HTML документе. Вставляется содержание перед (**:before**) или после (**:after**) какого либо элемента с помощью свойства **content**, которое собственно и определяет содержимое для вставки.

p:after {content: " Text!"; }

Селекторы

Стилевые правила записываются в своём формате, отличном от HTML. Основным понятием выступает **селектор** – это некоторое имя стиля, для которого добавляются параметры форматирования. В качестве селектора выступают элементы, классы и идентификаторы

```
селектор      свойство      значение
└──┬────────┬────────┬────────┘
body { background: #ffc910; }
```

Вначале пишется имя селектора, например, `body`, это означает, что все стилевые параметры будут применяться к элементу `body`, затем идут фигурные скобки, в которых записывается стилевое свойство, а его значение указывается после двоеточия. Стилевые свойства разделяются между собой точкой с запятой, в конце этот символ можно опустить

Обратите внимание, CSS не чувствителен к регистру, переносу строк, пробелам и символам табуляции, поэтому форма записи зависит от желания разработчика

Селекторы. Правила применения стилей

Существуют правила, которые необходимо знать при описании стиля

Форма записи

Для селектора допускается добавлять каждое стилевое свойство и его значение по отдельности:

```
td { background: olive; }  
td { color: white; }  
td { border: 1px solid black; }
```

Однако такая запись не очень удобна. Приходится повторять несколько раз один и тот же селектор, да и легко запутаться в их количестве. Поэтому все свойства для каждого селектора лучше писать вместе:

```
td {  
    background: olive;  
    color: white;  
    border: 1px solid black;  
}
```

Эта форма записи более наглядная и удобная в использовании

Селекторы. Правила применения стилей

Чем ниже значение, тем выше приоритет

Если для селектора вначале задаётся свойство с одним значением, а затем то же свойство, но уже с другим значением, то применяться будет то значение, которое в коде установлено ниже:

```
p { color: green; }  
p { color: red; }
```

Для селектора p цвет текста вначале установлен зелёным, а затем красным. Поскольку значение red расположено ниже, то оно в итоге и будет применяться к тексту

Обратите внимание, на самом деле такой записи лучше вообще избегать и удалять повторяющиеся значения. Но подобное может произойти случайно, например, в случае подключения разных стилевых файлов, в которых содержатся одинаковые селекторы

Селекторы. Правила применения стилей

Значения

У каждого свойства может быть только соответствующее его функции значение. Например, для color, который устанавливает цвет текста, в качестве значений недопустимо использовать одно число

Комментарии

Комментарии нужны, чтобы делать пояснения по поводу использования того или иного стилевого свойства, выделять разделы или писать свои заметки. Комментарии позволяют легко вспоминать логику и структуру селекторов, и повышают разборчивость кода. **Обратите внимание**, вместе с тем, добавление текста увеличивает объём документов, что отрицательно сказывается на времени их загрузки. Поэтому комментарии обычно применяют в отладочных или учебных целях, а при выкладывании сайта в сеть их стирают

Чтобы пометить, что текст является комментарием, применяют следующую конструкцию `/* Текст комментария */`

Селекторы. Классы

Классы применяют, когда необходимо определить стиль для индивидуального элемента Web-страницы или задать разные стили для одного элемента:

```
p.c1 {  
    color: red;  
}  
<p class="c1">Текст</p>
```

Текст

Внутри стиля вначале пишется желаемый элемент, а затем, через точку пользовательское имя класса. Имена классов должны начинаться с латинского символа и могут содержать в себе символ дефиса (-) и подчеркивания (_). Использование русских букв в именах классов недопустимо. Чтобы указать в коде HTML, что элемент используется с определённым классом, к элементу добавляется атрибут `class="имя_класса"`

Селекторы. Классы

Можно, также, использовать классы и без указания элемента:

```
.c1 {  
    color: red;  
}  
<h1 class="c1">Заголовок</h1>  
<p class="c1">Текст</p>
```

При такой записи, стиль применится ко всем элементам с заданным классом



Заголовок

Текст

Селекторы. Использование разных классов

К любому элементу одновременно можно добавить несколько классов, перечисляя их в атрибуте class через пробел. В этом случае к элементу применяется стиль, описанный в правилах для каждого класса:

```
.c1 {  
    color: red;  
}
```

```
.c2 {  
    background-color: blue;  
}
```



```
<h1 class="c1 c2">Заголовок</h1>
```

```
<p class="c1 c2">Текст</p>
```

Обратите внимание, в стилях также допускается использовать запись вида `.layer1.layer2`, где `layer1` и `layer2` представляют собой имена классов. Стиль применяется только для элементов, у которых одновременно заданы классы `layer1` и `layer2`

Селекторы. Идентификаторы

Идентификатор определяет уникальное имя элемента, которое используется для изменения его стиля и обращения к нему через скрипты:

```
#p1 {  
    color: blue;  
}
```

```
<p id="p1">Текст</p>
```

Текст

При описании идентификатора вначале указывается символ решётки #, затем идет имя идентификатора. Оно должно начинаться с латинского символа и может содержать в себе символ дефиса (-) и подчеркивания (_). Использование русских букв в именах идентификатора недопустимо. В отличие от классов идентификаторы должны быть уникальны, иными словами, встречаться в коде документа только один раз

Обращение к идентификатору происходит аналогично классам, но в качестве ключевого слова у элемента используется атрибут id, значением которого выступает имя идентификатора. Символ решётки при этом уже не указывается

Селекторы. Комбинаторы: вложенные

При создании Web-страницы часто приходится вкладывать одни элементы внутрь других. Чтобы стили для этих элементов использовались корректно, помогут селекторы, которые работают только в определённом контексте. Таким образом можно одновременно установить стиль для отдельного элемента, а также для элемента, который находится внутри другого. **Вложенный** селектор состоит из простых селекторов разделённых пробелом. Синтаксис следующий:

селектор1 селектор2 { Описание правил стиля }

В этом случае стиль будет применяться к селектор2 когда он размещается внутри селектор1:

```
p strong {  
    color: blue;  
}
```

<p>Текст Текст</p>

Текст *Текст*

Обратите внимание, не обязательно контекстные селекторы содержат только один вложенный селектор. В зависимости от ситуации допустимо применять два и более последовательно вложенных друг в друга селекторов:

```
p em strong {  
    color: blue;  
}
```

Текст *Текст*

Селекторы. Комбинаторы: дочерние

Дочерним селектором считается такой, который в дереве элементов находится прямо внутри родительского элемента. Синтаксис следующий:

селектор1 > селектор2 { Описание правил стиля }

Стиль применяется к селектор2, но только в том случае, если он является дочерним для селектор1:

```
.c1 > p {  
    color: red;  
}  
  
<div class="c1">  
    <p>Текст</p>  
    <div>  
        <p>Текст</p>  
    </div>  
</div>
```



По своей логике дочерние селекторы похожи на селекторы контекстные. Разница между ними следующая. Стиль к дочернему селектору применяется только в том случае, когда он является прямым потомком, иными словами, непосредственно располагается внутри родительского элемента. Для контекстного селектора же допустим любой уровень вложенности

Селекторы. Комбинаторы: соседние

Соседними называются элементы Web-страницы, когда они следуют непосредственно друг за другом в коде документа. Для управления стилем соседних элементов используется символ плюса **+**, который устанавливается между двумя селекторами. Синтаксис следующий:

селектор1 + селектор2 { Описание правил стиля }

Стиль при такой записи применяется к селектор2, но только в том случае, если он является соседним для селектор1 и следует сразу после него:

```
.c1 + .c2 {  
    color: blue;  
}
```

```
<p>
```

```
    <em class="c1">Текст</em>
```

```
    <em class="c2">Текст</em>
```

```
    <em class="c2">Текст</em>
```

```
</p>
```

<i>Текст Текст Текст</i>

Селекторы. Комбинаторы: смежные

Смежный селектор позволяет выбрать элементы, которые будут отобраны на основе их родственных элементов, т.е. те, у которых один и тот же общий родитель. Он создается с помощью символа тильды ~ между двумя элементами внутри селектора. Первый элемент определяет, что второй элемент должен быть родственным с ним, и у обоих должен быть один и тот же родитель:

селектор1 ~ селектор2 { Описание правил стиля }

Стиль при такой записи применяется к селектор2, которые следуют после любых элементов селектор1:

```
#id1 ~ em {  
    color: red;  
}  
  
<p>  
    <b id="id1">Текст</b>  
    <em>Текст</em>  
    <span>Текст</span>  
    <em>Текст</em>  
</p>
```

Текст <i>Текст</i> Текст <i>Текст</i>
--

Селекторы на основе атрибутов

Многие элемент различаются по своему действию в зависимости от того, какие в них используются атрибуты. Чтобы гибко управлять стилем подобных элементов, в CSS введены селекторы атрибутов. Они позволяют установить стиль по присутствию определённого атрибута элемента или его значения

Простой селектор атрибута

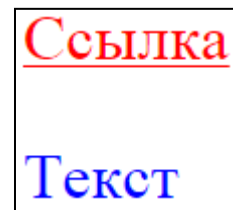
Устанавливает стиль для элемента, если задан специфичный атрибут элемента. Его значение в данном случае не важно. Синтаксис следующий:

[атрибут] { Описание правил стиля }

Селектор[атрибут] { Описание правил стиля }

Пример:

```
[href] { color: red; }  
p[title] { color: blue; }  
<a href="#">Ссылка</a>  
<p title="text">Текст</p>
```



Селекторы на основе атрибутов

Атрибут со значением

Устанавливает стиль для элемента в том случае, если задано определённое значение специфичного атрибута. Синтаксис следующий:

`[атрибут="значение"] { Описание правил стиля }`

`Селектор[атрибут="значение"] { Описание правил стиля }`

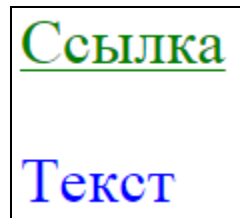
Пример:

```
[hreflang="en-US"] { color: green; }
```

```
p[title="text"] { color: blue; }
```

```
<a href="#" hreflang="en-US">Ссылка</a>
```

```
<p title="text">Текст</p>
```



Селекторы на основе атрибутов

Значение атрибута начинается с определённого текста

Устанавливает стиль для элемента в том случае, если значение атрибута элемента начинается с указанного текста. Синтаксис следующий:

[атрибут^="значение"] { Описание правил стиля }

Селектор[атрибут^="значение"] { Описание правил стиля }

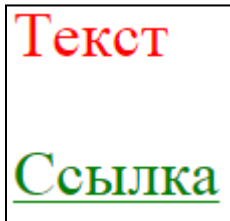
Пример:

```
[title^="te"] { color: red; }
```

```
a[href^="http://"] { color: green; }
```

```
<p title="text">Текст</p>
```

```
<a href="http://example.com">Ссылка</a>
```



Селекторы на основе атрибутов

Значение атрибута оканчивается определённым текстом

Устанавливает стиль для элемента в том случае, если значение атрибута оканчивается указанным текстом. Синтаксис следующий:

[атрибут\$="значение"] { Описание правил стиля }

Селектор[атрибут\$="значение"] { Описание правил стиля }

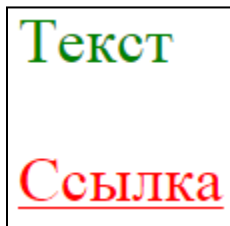
Пример:

```
[title$="xt"] { color: green; }
```

```
a[href$=".com"] { color: red; }
```

```
<p title="text">Текст</p>
```

```
<a href="http://example.com">Ссылка</a>
```



Селекторы на основе атрибутов

Значение атрибута содержит указанный текст

Возможны варианты, когда стиль следует применить к элементу с определённым атрибутом, при этом частью его значения является некоторый текст. При этом точно не известно, в каком месте значения включен данный текст – в начале, середине или конце. В подобном случае следует использовать синтаксис:

```
[атрибут*="значение"] { Описание правил стиля }
```

```
Селектор[атрибут*="значение"] { Описание правил стиля }
```

Пример:

```
[title*="ex"] { color: blue; }
```

```
a[href*="example"] { color: red; }
```

```
<p title="text">Текст</p>
```

```
<a href="http://example.com">Ссылка</a>
```

Текст

Ссылка

Селекторы на основе атрибутов

Одно из нескольких значений атрибута

Некоторые значения атрибутов могут перечисляться через пробел, например имена классов. Чтобы задать стиль при наличии в списке требуемого значения применяется следующий синтаксис:

```
[атрибут~="значение"] { Описание правил стиля }
```

```
Селектор[атрибут~="значение"] { Описание правил стиля }
```

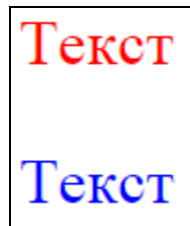
Пример:

```
[class~="c1"] { color: red; }
```

```
p[class~="c3"] { color: blue; }
```

```
<p class="c1 c2">Текст</p>
```

```
<p class="c2 c3">Текст</p>
```



Селекторы на основе атрибутов

Дефис в значении атрибута

В именах идентификаторов и классов разрешено использовать символ дефиса (-), что позволяет создавать значащие значения атрибутов id и class. Для изменения стиля элементов, в значении которых применяется дефис, следует воспользоваться следующим синтаксисом:

```
[атрибут|"значение"] { Описание правил стиля }
```

```
Селектор[атрибут|"значение"] { Описание правил стиля }
```

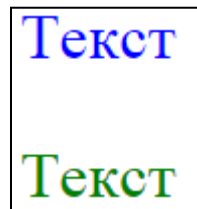
Пример:

```
[class|"my1"] { color: blue; }
```

```
p[class|"my2"] { color: green; }
```

```
<p class="my1-class">Текст</p>
```

```
<p class="my2-class">Текст</p>
```



Селекторы на основе атрибутов

Все перечисленные методы можно комбинировать между собой, определяя стиль для элементов, которые содержат два и более атрибута. В подобных случаях квадратные скобки идут подряд. Синтаксис следующий:

```
[атрибут1="значение1"] [атрибут2="значение2"] { Описание правил стиля }
```

```
Селектор[атрибут1="значение1"] [атрибут2="значение2"] {  
Описание правил стиля }
```

Пример:

```
[class="c1"] [title] { color: green; }  
p[class="c2"] [title="text"] { color: red; }  
<p class="c1" title="myTitle">Текст</p>  
<p class="c2" title="text">Текст</p>
```



Универсальный селектор

Иногда требуется установить одновременно **один стиль для всех элементов** Web-страницы, например, задать шрифт или начертание текста. В этом случае поможет универсальный селектор, который соответствует любому элементу Web-страницы. Для обозначения универсального селектора применяется символ звёздочки * и в общем случае синтаксис будет следующий:

* { Описание правил стиля }

Пример:

```
* { color: blue; }
```

```
<p>Текст</p>
```

Текст

Обратите внимание, в некоторых случаях указывать универсальный селектор не обязательно. Так, например, записи *.class и .class являются идентичными по своему результату

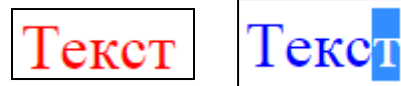
Селекторы. Псевдоклассы

Псевдоклассы определяют динамическое состояние элементов, которое изменяется с помощью действий пользователя, а также положение в дереве документа. Примером такого состояния служит текстовая ссылка, которая меняет свой цвет при наведении на неё курсора мыши. При использовании псевдоклассов браузер не перегружает текущий документ, поэтому с помощью псевдоклассов можно получить разные динамические эффекты на странице. Синтаксис следующий:

```
Селектор:Псевдокласс { Описание правил стиля }  
:Псевдокласс { Описание правил стиля }
```

Пример:

```
p { color: red; }  
p:hover { color: blue; }  
<p>Текст</p>
```



Вначале указывается селектор, к которому добавляется псевдокласс, затем следует двоеточие, после которого идёт имя псевдокласса. Допускается применять псевдоклассы к именам идентификаторов или классов, а также к контекстным селекторам. **Обратите внимание**, если псевдокласс указывается без селектора впереди, то он будет применяться ко всем элементам документа

Селекторы. Псевдоклассы

Некоторые псевдоклассы:

- **:link** – отвечает за стили не посещенной ссылки
- **:hover** – состояние объекта (не обязательно ссылки) при наведении на него мышкой
- **:active** – состояние активного объекта (например, для ссылки и зажатие ее мышкой)
- **:visited** – состояние посещенной ссылки
- **:focus** – когда используется какой-то объект на страницы, то на нем устанавливается фокус (в случае и текстовым поле это постановка курсора в это поле)
- **:first-child** – первый дочерний элемент текущего элемента
- **:last-child** – последний дочерний элемент текущего элемента

Селекторы. Псевдоэлементы

Псевдоэлементы – это особый вид свойств CSS, которые позволяют работать не над самим элементом, а над его отдельной частью. В CSS3 псевдоэлементы начинаются с двух двоеточий ::

Синтаксис следующий:

Селектор :: Псевдоэлемент { Описание правил стиля }

Пример:

```
p::first-letter { color: red; }
```

```
<p>Текст</p>
```

Текст

Вначале следует имя селектора, затем пишется двоеточие, после которого идёт имя псевдоэлемента. Каждый псевдоэлемент может применяться только к одному селектору, если требуется установить сразу несколько псевдоэлементов для одного селектора, правила стиля должны добавляться к ним по отдельности

Селекторы. Псевдоэлементы

Список псевдоэлементов:

- **::after** – используется для вывода желаемого контента после элемента, к которому он добавляется
- **::before** – применяется для отображения желаемого контента до элемента, к которому он добавляется
- **::first-letter** – определяет стиль первого символа в тексте элемента, к которому добавляется
- **::first-line** – задает стиль первой строки форматированного текста
- **::selection** – применяет стиль к выделенному пользователем фрагменту текста

Селекторы. Группирование

При создании стиля для сайта, когда одновременно используется множество селекторов, возможно появление повторяющихся стилевых правил. Чтобы не повторять дважды одни и те же элементы, их можно сгруппировать для удобства представления и сокращения кода

Селекторы группируются в виде списка селекторов, разделенных между собой запятыми. Синтаксис следующий:

Селектор 1, Селектор 2, ... Селектор N { Описание правил
стиля }

Пример:

```
h1, h2, h3 { font-family: Arial; }  
h1 { color: red; }  
h2 { color: green; }  
h3 { color: blue; }
```

При такой записи правила стиля применяются ко всем селекторам, перечисленным в одной группе

Селекторы. Полезные советы: порядок элементов

При построении CSS будьте логичны, соблюдайте "значимость" элементов и их порядок, так же как они вложены друг в друга в HTML коде

Например:

```
body      { сначала опишите стиль страницы в целом }  
div       { потом её отдельных частей – блоков }  
a         { затем ссылок }  
h1 – h6  { далее заголовков }  
p         { и в конце параграфов }
```

Для чего это нужно?

- Просто для удобного чтения и "навигации" по CSS описанию. Когда потребуется найти какой-нибудь элемент, уже изначально будет представление где он приблизительно находится, в начале, середине, или конце
- Загрузка страницы происходит не моментально и не всегда приятно наблюдать как содержание данной страницы при загрузке "прыгает" и всячески "шевелится" так как сначала прописываются "малозначимые" стили элементов, например шрифт параграфов, а в конце "значительные" например размеры блоков, с помощью которых свёрстан весь сайт. К тому же загрузка, по каким либо причинам, вообще может пройти не до конца

Селекторы. Полезные советы: система именований

При использовании классов и идентификаторов придумывайте им осмысленные информативные имена

Варианты `.aaa` `.a12` `#abc` `#cba` приведут к путанице, а также возможно в Вашем коде будет разбираться посторонний человек

Придумайте свою "систему" названий и не нарушайте её, так сэкономите собственное время и затраченные усилия

Наследование

При указании стиля для элемента часть свойств может быть унаследована его дочерними элементами и потомками, этот эффект называется **наследованием**.

Например, все элементы расположенные внутри элемента `body` являются его дочерними элементами и потомками. Если в стиле для `body` задать с помощью свойства `color` красный цвет текста, то цвет текста всех его дочерних элементов и потомков тоже станет красным

Наследуемые свойства можно переопределить, применив индивидуальное правило для нужного элемента

Чтобы узнать, какие CSS-свойства наследуются, а какие нет, нужно посмотреть описание конкретного свойства в CSS-справочнике (например: <https://webref.ru/css>)

Специфичность

Специфичность селекторов определяет их приоритетность в таблице стилей. Чем специфичнее селектор, тем выше его приоритет. Для вычисления специфичности селектора используются три группы чисел.

Специфичность = А В С

А — количество селекторов по id

В — количество селекторов по class, атрибутов и псевдоклассов

С — количество типов элементов

Расчёт производится следующим образом:

- Считается число идентификаторов в селекторе (=a)
- Считается число селекторов классов, атрибутов и псевдоклассов в селекторе (=b)
- Считается число селекторов типа и псевдоэлементов в селекторе (=c)
- Селектор внутри псевдокласса отрицания (:not) считается как любой другой селектор, но сам псевдокласс отрицания не участвует в вычислении селектора
- Универсальный селектор (*) и комбинаторы не участвуют в вычислении веса селектора

Специфичность

В примере селекторы расположены в порядке увеличения их специфичности:

1. *	/* a=0 b=0 c=0 -> специфичность = 0 0 0 */
2. li	/* a=0 b=0 c=1 -> специфичность = 0 0 1 */
3. ul li	/* a=0 b=0 c=2 -> специфичность = 0 0 2 */
4. ul ol+li	/* a=0 b=0 c=3 -> специфичность = 0 0 3 */
5. h1 + *[rel=up]	/* a=0 b=1 c=1 -> специфичность = 0 1 1 */
6. ul ol li.red	/* a=0 b=1 c=3 -> специфичность = 0 1 3 */
7. li.red.level	/* a=0 b=2 c=1 -> специфичность = 0 2 1 */
8. #x34y	/* a=1 b=0 c=0 -> специфичность = 1 0 0 */
9. #s12:not (p)	/* a=1 b=0 c=1 -> специфичность = 1 0 1 */

Стиль для элемента, определённый внутри атрибута style, имеет больший приоритет, чем любой селектор, определённый в таблице стилей. Однако, если для конкретного свойства в таблице стилей указать специальное объявление **!important**, то оно будет иметь больший приоритет, чем значение аналогичного свойства, указанного в атрибуте style

Специфичность и !important

Синтаксис применения !important следующий:

Свойство: значение !important

Пример:

```
* {  
    color: blue !important;  
}
```

Вначале пишется желаемое стилевое свойство, затем через двоеточие его значение и в конце после пробела указывается ключевое слово !important

Каскадность

Каскадность это фундаментальная особенность CSS, с помощью которой браузер определяет значения каких свойств будут применены к элементу при возникновении конфликта свойств. Конфликт свойств возникает, когда для одного элемента *определено более одного правила с одинаковым приоритетом* и они содержат одинаковые свойства, но с разными значениями.

Каскадность работает следующим образом: если в таблице стилей для одного элемента определено несколько правил, селекторы которых имеют одинаковую специфичность и они содержат конфликтующие свойства то, для элемента устанавливаются значения тех свойств, которые расположены ниже в таблице стилей.

Если разные правила для одного элемента содержат свойства, которые не конфликтуют, то они объединяются в один стиль, т.е. каждое новое правило добавляет новую информацию о стиле к тому правилу, которое находится перед ним.

Закрепление: простые селекторы, псевдоэлементы

селектор по типу элемента	h1
универсальный селектор	*
селектор по id	#menu
селектор по class	.special
псевдоклассы	:link, :visited :hover, :active, :focus :first-child
селекторы атрибутов	[title], [alt="Пушкин"], [href^="http://"], [href\$=".pdf"]
последовательность простых селекторов	a.external:hover, .main#menu
псевдоэлементы	::first-letter, ::first-line ::before, ::after

Закрепление: комбинаторы

Обозначение	Описание	Пример
пробел	элемент-потомок <i>Descendant</i>	ul li #main p
>	дочерний элемент <i>Child</i>	p > strong body > *
+	ближайший последующий смежный элемент <i>Adjacent Sibling</i>	h1 + p li + li
~	любой последующий смежный элемент <i>General Sibling</i>	h2 ~ p
,	группировка селекторов (не комбинатор)	td, th ul li, td#item

Стили: закругленные углы

1. #1 { border-radius:10px; }
2. #2 { border-radius:50%; }
3. #3 { border-radius:25px 5px; }
4. #4 { border-radius:40px 30px 20px 10px; }



Стили: добавление тени к блочным элементам

- #el1 { box-shadow:4px 4px black; }
- #el2 { box-shadow:6px 6px 6px 2px black; }
- #el3 { box-shadow:0px 0px 6px 2px black inset; }

1. Первые два значения свойства *box-shadow* задают величину смещения тени по горизонтали и вертикали в пикселях, а третье значение задает цвет тени.

box-shadow:4px 4px black;

2. Также данное свойство может иметь значения указывающие радиус разброса тени (третье значение) и размер тени (четвертое значение).

box-shadow:6px 6px 6px 2px black;

3. С помощью значения *inset* Вы можете указать, что тень должна быть не внешней, а внутренней. Элементы с внутренними тенями кажутся 'вдавленными'.

box-shadow:0px 0px 6px 2px black inset;

Верстка

- Процесс формирования документа через набор кода или с помощью WYSIWYG-редакторов
- Верстка в веб обычно представляет собой реализацию разработанного дизайна в виде HTML/CSS кода
- Современная вёрстка должна отвечать стандартам HTML5 и CSS3

Виды вёрстки: табличная и блочная

Виды вёрстки: табличная вёрстка

- Таблица состоит из строк и столбцов, которые образуют ячейки
- При табличной вёрстке модульная сетка (разметка структуры) создается ячейками (строками и столбцами) таблицы
- Позиционирование элементов страницы осуществляется через помещение в ячейки

Табличная вёрстка

- В HTML вывод таблицы осуществляется с помощью тега `<table>`. Вложенный тег `<tr>` формирует строку таблицы. В неё вкладывается тег `<td>`, который формирует ячейку в данной строке, т.е. столбец.

```
<table>
```

```
  <tr>
```

```
    <td> Ячейка 1 </td>
```

```
    <td> Ячейка 2 </td>
```

```
    <td> Ячейка 3 </td>
```

```
  </tr>
```

```
  <tr>
```

```
    <td> Ячейка 4 </td>
```

```
    <td> Ячейка 5 </td>
```

```
    <td> Ячейка 6 </td>
```

```
  </tr>
```

```
</table>
```

Ячейка 1	Ячейка 2	Ячейка 3
Ячейка 4	Ячейка 5	Ячейка 6

Табличная вёрстка

- Количество столбцов в каждой строке должно быть одинаковым (но можно объединять ячейки по вертикали и горизонтали с помощью атрибутов `rowspan="число"` и `colspan="число"` тега `<td>`)

```
<table>
```

```
<tr>
```

```
<td rowspan="2"> Ячейка 1 и 4 </td>
```

```
<td colspan="2"> Ячейка 2 и 3 </td>
```

```
</tr>
```

```
<tr>
```

```
<td> Ячейка 5 </td>
```

```
<td> Ячейка 6 </td>
```

```
</tr>
```

```
</table>
```

Ячейка 1 и 4	Ячейка 2 и 3	
	Ячейка 5	Ячейка 6

Табличная вёрстка

- В атрибутах ячейки (`<td></td>`) или соответствующем селекторе CSS можно указывать:
- Выравнивание содержимого по горизонтали (`align=""`) и вертикали (`valign=""`)
- Ширину (`width=""`) и высоту (`height=""`) ячейки. При этом, ячейки обладают одинаковой шириной по столбцу и одинаковой высотой по строке, т.е. **зависят друг от друга**

Табличная вёрстка

- В каждую ячейку можно вкладывать новую таблицу или любой другой блочный элемент
- Можно настраивать различные свойства ячеек (отступы, границы, поля) и вложенных элементов с помощью CSS

Табличная вёрстка: плюсы и минусы

Преимущества:

- Простота по сравнению с блочной
- Кроссбраузерность

Недостатки:

- Громоздкий код
- Недостаточная гибкость

Виды вёрстки: блочная вёрстка

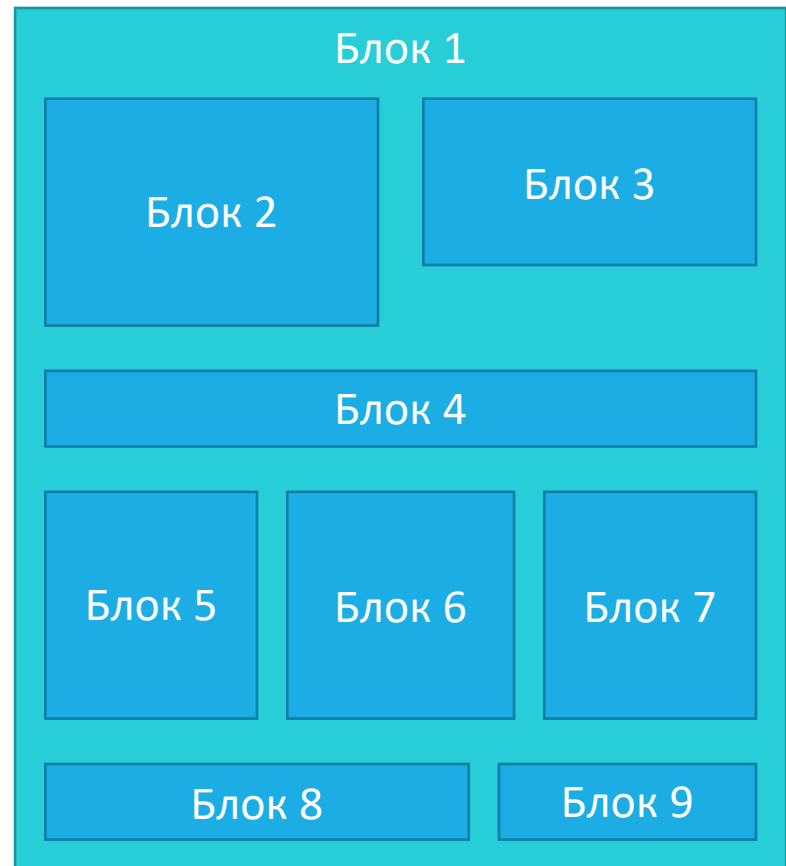
- Разметка страницы осуществляется с помощью универсальных блочных элементов `<div></div>` (слоёв или блоков).
- При этом HTML-код содержит только теги разметки и логического форматирования, а всё визуальное оформление выносится в CSS.
- Таблицы используются только для вывода табличных данных.

Блочная вёрстка

- `<div>` представляет собой прямоугольную область.
- Блоку присваивается класс из таблицы CSS.
- В CSS настраивается позиционирование блока, выравнивание относительно других элементов, строчность\блочность*, поля, отступы, границы, фон, выравнивание и стили содержимого.

Блочная вёрстка

- Габариты каждого блока не зависят от габаритов любого другого
- Можно выводить блоки из потока документа с помощью позиционирования в CSS
- В `<div>` можно вкладывать любые строчные или блочные элементы в т.ч. другие `<div>`



Блочная вёрстка: плюсы и минусы

Преимущества:

- Более компактный код
- Намного больше возможностей, гибкость
- Блоки загружаются быстрее таблиц

Недостатки:

- Требуется больше знаний и навыков

<table> против <div>

	Таблица	div
Колонки	Колонки формируются ячейками таблицы, их высота одинакова и взаимосвязана.	Колонки создаются разными слоями, их высота разная и зависит от содержания.
Ширина	Если ширина таблицы явно не указана, она вычисляется на основе содержимого таблицы.	По умолчанию слой занимает всю доступную ему ширину.
Расположение	Строки таблицы отображаются в том порядке , как они представлены в коде. Ячейки идут слева направо и сверху вниз.	Порядок слоёв в коде может не соответствовать их положению в браузере.
Загрузка	Как правило, пока таблица не загрузится полностью , содержимое её не будет показываться. Если на веб-странице размещена большая таблица, загрузка страницы существенно замедляется.	Слои отображаются последовательно , по мере загрузки документа.
HTML-код	Код для создания таблиц может быть громоздок , особенно если требуется объединить множество ячеек или сделать несколько вложенных таблиц.	Код, как правило, компактный .

Виды вёрстки (дизайна)

- Фиксированная
- Резиновая
- Адаптивная\отзывчивая

Фиксированная вёрстка

- Размеры всех элементов жестко определены (заданы в пикселях)
- При изменении разрешения (размера окна браузера), размер элементов в пикселях не меняется
- Простота перевода дизайна в код
- Некорректное отображение при изменении разрешения

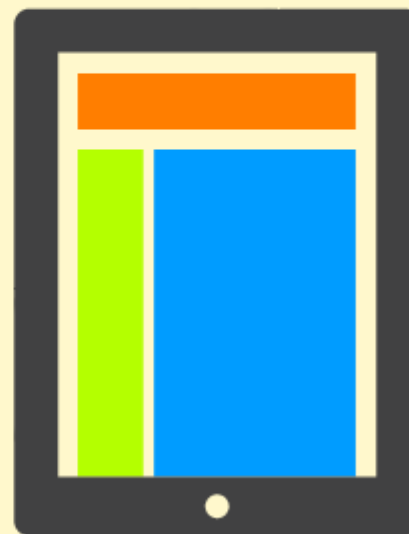
Фиксированная вёрстка



Резиновая вёрстка

- Размеры всех элементов заданы в процентах от разрешения (размера окна)
- При изменении разрешения (размера окна браузера), размер элементов изменяется пропорционально
- Более-менее подстраивается под размер экрана
- Сложнее корректно реализовать дизайн в коде.

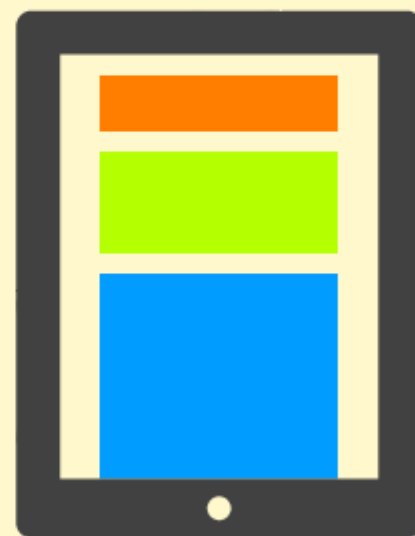
Резиновая вёрстка



Адаптивная вёрстка

- Адаптируется к набору разрешений (прыгает по контрольным точкам). Размеры элементов указываются в пикселях, при этом значения изменяются в зависимости от разрешения.
- Подстраивается под любое устройство и размер экрана (с нюансами). Позволяет избавиться от «мобильных версий»
- Гораздо сложнее фиксированной и резиновой

Адаптивная вёрстка



Отзывчивая вёрстка

- Приспосабливается к любому разрешению. Размеры элементов указываются в относительных единицах.
- При изменении разрешения (размера окна браузера), размер элементов и их положение изменяется так, как это необходимо для корректного отображения.
- Подстраивается под любое устройство и размер экрана. Позволяет избавиться от «мобильных версий».
- Гораздо сложнее фиксированной и резиновой.

Отзывчивая вёрстка

